

Modélisation avancée des Systèmes d'Information

05 - Observer

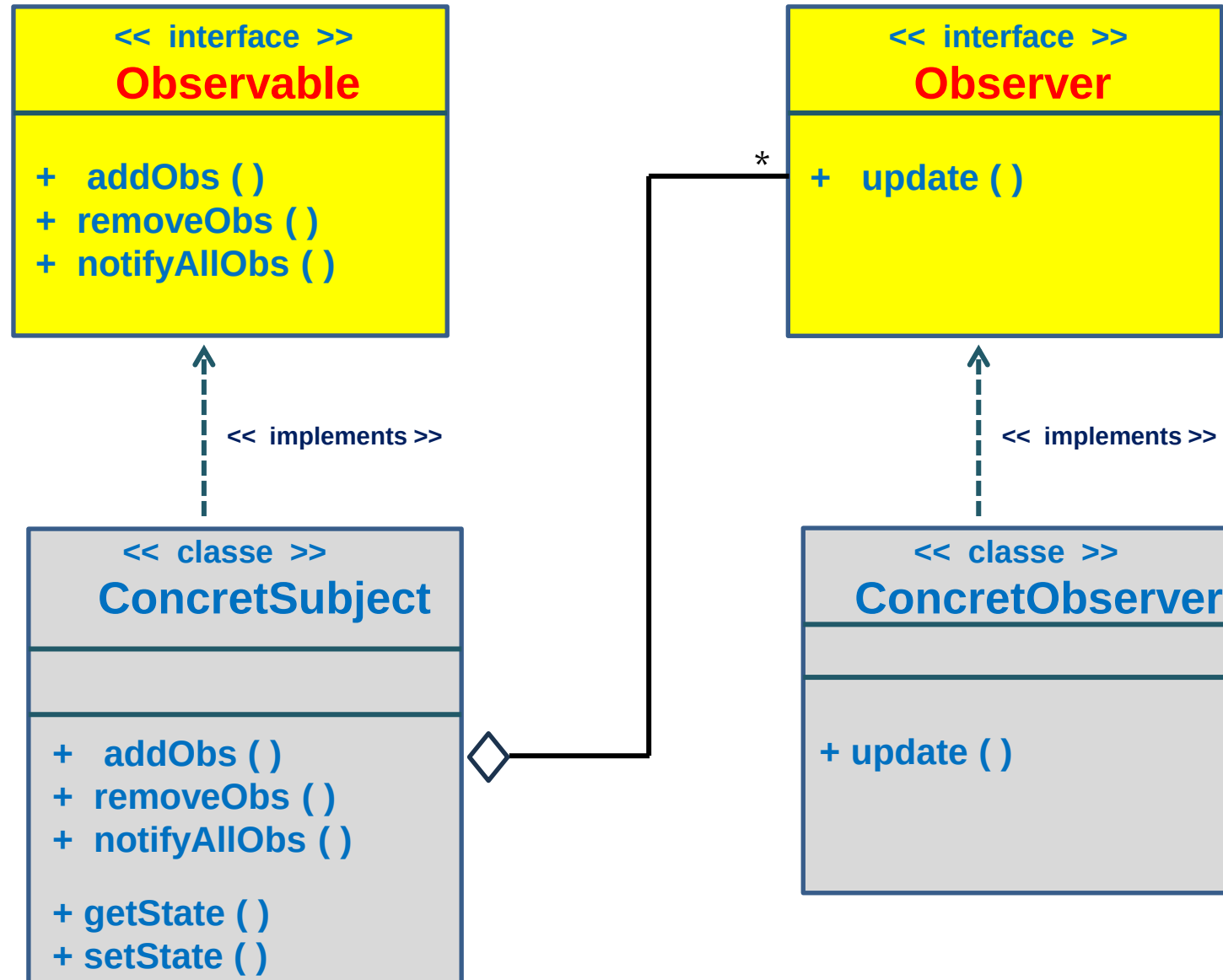
Mohamed ZAYANI

2025/2026

Définition d'Observer

- Le patron de conception Observer est **un patron comportemental** qui définit une relation **"un-à-plusieurs"** entre des objets: Lorsqu'un objet (appelé **Sujet** ou **Observable**) change d'état, tous ses **Observateurs** (ou **Observers**) en sont automatiquement informés et mis à jour.
- **Objectif principal:**
 - ❖ découpler l'objet principal (le sujet) des objets qui dépendent de lui (les observateurs), favorisant ainsi une meilleure extensibilité et maintenabilité.
- **Problématique:**
 - Comment notifier automatiquement les objets dépendants sans les coupler fortement au sujet ?
 - Comment garantir la cohérence des données entre les objets interdépendants ?.
- **Solution:**
 - Le **Sujet** conserve une liste d'observateurs et fournit des méthodes pour ajouter, supprimer et notifier ces observateurs.
 - Les **Observateurs** s'abonnent au sujet et sont automatiquement informés des changements d'état.
 - **L'objet sujet informe automatiquement ses observateurs sans avoir besoin de connaître leur implémentation**

Modélisation UML



Implémentation JAVA

```
public interface Observer
{
    void update (String message);
}
```

```
public class ConcretObserver
implements Observer {
    private String libelle;

    public ConcretObserver(String libelle) {
        super();
        this.libelle = libelle;
    }
    @Override
    public void update(String message) {
        System.out.println(libelle + " a reçu le
nouveau message : " + message);
    }
}
```

```
public interface Observable {
    void addObserver(Observer o);
    void removeObserver (Observer o);
    void notifyAllObservers();
}
```

```
import java.util.ArrayList;
import java.util.List;

public class Data implements Observable {
    private List<Observer> observateurs = new ArrayList<>();
    private String message;
    public void addObserver(Observer o) {
        observateurs.add(o);
    }
    public void removeObserver(Observer o) {
        observateurs.remove(o);
    }
    public void notifyAllObservers() {
        for (Observer o : observateurs)
        {
            o.update(message);
        }
    }
    public String getMessage() {
        return message;
    }
    public void setMessage(String message) {
        this.message = message;
        notifyAllObservers();
    }
}
```

Implémentation JAVA

```
public class Client {  
    public static void main(String[] args) {  
        // créer un sujet concret (de type Data)  
        Data data = new Data();  
  
        // créer trois observateurs concrets  
        Observer obs1 = new ConcretObserver("Observateur 1");  
        Observer obs2 = new ConcretObserver("Observateur 2");  
        Observer obs3 = new ConcretObserver("Observateur 3");  
  
        //enregistrer les observateurs  
        data.addObserver(obs1);  
        data.addObserver(obs2);  
        data.addObserver(obs3);  
  
        //changer la valeur du message  
        data.setMessage("message 1");  
  
        // supprimer le deuxième observateur  
        data.removeObserver(obs2);  
  
        System.out.println("-----");  
        // changer une deuxième fois la valeur du message  
        data.setMessage("message 2");  
    }  
}
```

Résultat:

```
Observateur 1 a reçu le nouveau message : message 1  
Observateur 2 a reçu le nouveau message : message 1  
Observateur 3 a reçu le nouveau message : message 1  
-----  
Observateur 1 a reçu le nouveau message : message 2  
Observateur 3 a reçu le nouveau message : message 2
```

Cas d'utilisation du Patron Observer

- **Interfaces Graphiques (GUI)** : Actualisation de composants graphiques lors d'un événement (boutons, champs de texte).
- **Flux de données en temps réel** : Notifications des abonnés dans les réseaux sociaux, systèmes de messagerie.
- **Systèmes de surveillance** : Capteurs de température, systèmes de sécurité.
- **Gestion des événements** : Listeners dans les applications web et mobiles.